AAI-540 Machine Learning Operations

Applied Artificial Intelligence

University of San Diego

Housing Price Prediction: An End-to-End MLOps System

Course Project Report

Submitted to: Dr. Raj Kumar Garg

Group members:
Hemlatha Kaur Saran
Bharath T S
Zoher Sabunwala

Author Note

This report documents the methodology, implementation, evaluation, and production design of a supervised regression system that predicts California census block-group median house values, together with an AWS SageMaker-oriented MLOps architecture (Feature Store, Model Registry, monitoring, and gated CI/CD). Modelling results are taken from the executed project notebooks in the GitHub repository. SageMaker components describe the target production design unless explicitly marked as implemented.

ABSTRACT

This project investigates neighbourhood-level house-price estimation as a supervised regression problem and designs the surrounding machine-learning operations (MLOps) system needed to train, serve, monitor, and safely retrain the model. The data source is the California Housing dataset from the 1990 U.S. census (20,640 block groups, 10 raw columns). A reproducible notebook pipeline covers data checks, exploratory analysis, feature engineering (three household ratios and three haversine city distances), an 80/20 income-stratified train/test split, six candidate models with 5-fold cross-validation, randomised hyperparameter search, and held-out evaluation. Tuned XGBoost achieved the best test performance: MAE $26,882 (13.0% of mean price), RMSE $41,441, and R^2 0.868, cutting MAE by 41% and RMSE by 33% relative to untuned linear regression. Error roughly doubles on the 179 test rows censored at $500,001. Beyond the model, the report specifies a SageMaker production design spanning versioned S3 ingestion, Feature Store, training and automatic model tuning, Model Registry with quality gates ($R^2 \geq 0.85$), real-time and batch serving, Model Monitor / Clarify, and CI/CD via SageMaker Pipelines.

Source code, notebooks, tests, and this design live at
https://github.com/bharath-ts/msaai-540-mlops.

Keywords: housing price prediction, California Housing, supervised regression, XGBoost, MLOps, AWS SageMaker, Feature Store, model monitoring, CI/CD.

TABLE OF CONTENTS

The outline below mirrors the Heading 1 / Heading 2 structure of this document.

In Microsoft Word: right-click here → Update Field to refresh page numbers. A manual outline is also listed below.

Manual outline (always visible)

1. Introduction

1.1 Problem Statement
1.2 Business Context and Stakeholders
1.3 Project Objectives
1.4 Goals, Non-Goals, Scope, and Assumptions
1.5 Repository and Artefacts

2. Dataset and Exploratory Analysis

2.1 Data Source and Dictionary
2.2 Data Quality and Ingestion Design
2.3 Exploratory Findings
2.4 Preprocessing Design

3. Feature Engineering

3.1 Engineered Feature Catalogue
3.2 Rationale and Leakage Control
3.3 Shared Feature Code

4. Model Development and Training

4.1 Candidate Models
4.2 Train/Validation Protocol
4.3 Hyperparameter Search
4.4 Cross-Validation Results

5. Evaluation and Results

5.1 Held-Out Test Performance
5.2 Success Criteria
5.3 Error Analysis and Feature Importance
5.4 Sample Prediction

6. ML System Architecture

6.1 End-to-End Workflow
6.2 Notebook versus SageMaker Mapping
6.3 Implemented versus Design-Only Components

7. Data Platform and Feature Store

8. Training Pipelines and Model Registry

9. Deployment (Real-Time and Batch)

10. Monitoring, Drift, and Alerts
11. Security and Governance
12. Continuous Integration and Delivery (CI/CD)
13. Discussion, Limitations, and Future Work
14. Conclusion
15. References
Appendices A–D

1. Introduction

Estimating the value of residential areas is slow, manual, and inconsistent. Analysts, lenders, and planners typically rely on spreadsheets, local expertise, and outdated comparables, which makes valuations hard to scale and hard to reproduce. This project asks whether a model can estimate the median house value of a California census block group accurately enough to serve as a repeatable first-pass valuation tool—and how that model should be operated as a production machine-learning system rather than a one-off notebook.

The system is framed as supervised regression on the California Housing dataset. Beyond accuracy, the goal is an operational ML system on AWS SageMaker: governed features, a reproducible training pipeline, a model registry, real-time and batch deployment, monitoring for data and model drift, and automated CI/CD so the model can be retrained and promoted safely as new data arrives. All modelling code and documentation for the course project are maintained at https://github.com/bharath-ts/msaai-540-mlops.

1.1 Problem Statement

Let each census block group be represented by a feature vector x that includes location (longitude, latitude, ocean proximity), housing stock attributes (median age, rooms, bedrooms), and demographics (population, households, median income), together with engineered ratios and city distances. The learning objective is to estimate a regression function f that maps x to median house value y (U.S. dollars). Predictions are intended for screening and prioritisation at the block-group level, not for appraising individual homes or making lending decisions.

Formally, the project treats the task as minimising expected prediction error on held-out block groups under MAE, RMSE, and R^2, while also designing the operational controls that keep a deployed model trustworthy over time.

1.2 Business Context and Stakeholders

Valuation questions arise for lenders screening regional portfolios, city planners comparing neighbourhoods, developers choosing where to build, and analysts producing market reports. Each needs a fast, consistent estimate with a known error range so human effort can focus on cases that require judgement.

Table 1. Stakeholders and intended use of model outputs

Stakeholder	Interest	How they use the output
Data analysts and planners	Fast first-pass valuations	Batch predictions for screening/ranking
Lenders and risk teams	Consistent, auditable estimates	Portfolio exposure; known error/bias reporting
ML and platform engineers	Reproducible, maintainable system	Operate pipeline, endpoint, monitoring, retraining
Compliance and security	Protection, fairness, traceability	Review checklist, model card, audit trail

Project sponsors	Evidence goals are met	Review success metrics

1.3 Project Objectives

- Develop a reproducible regression pipeline from raw California Housing data to trained models and held-out evaluation.
- Engineer leakage-safe features (household ratios and haversine distances) that improve predictive signal beyond raw counts.
- Compare linear, tree, and ensemble models under a common 5-fold cross-validation protocol; tune the strongest candidates.
- Meet stated success criteria: $R^2 \geq 0.85$, MAE $\leq$ 15% of mean price, and test metrics within a few percent of cross-validation.
- Report error separately for price-capped versus uncapped rows and document where the model is weakest.
- Design an AWS SageMaker production architecture covering Feature Store, Model Registry, serving, monitoring, and gated CI/CD.
- Publish code, tests, and design documentation in https://github.com/bharath-ts/msaai-540-mlops.

1.4 Goals, Non-Goals, Scope, and Assumptions

Goals: (1) prediction error less than 15% of mean house value at the block-group level; (2) transparent goals and assumptions; (3) honest accuracy reporting, including failure modes; (4) present the work as a repeatable, monitored, automatically tested ML system.

Non-goals: the system is not an appraiser; it does not forecast future prices; it does not make or suggest lending decisions; and it does not attempt to estimate values above the $500,001 censoring point in the training data.

Scope: target = median house value; batch and single-record prediction; geographic scope = California block groups represented by the nine raw predictors. Assumption: 1990 census relationships are a reasonable proxy until current data are used. Assumption: rows are treated as independent even though neighbouring block groups are spatially correlated (spatial cross-validation is future work). Trainability: 20,640 rows (~1.2 MB) make retraining inexpensive. Constraint: production components align with AWS SageMaker services.

1.5 Repository and Artefacts

Canonical repository: https://github.com/bharath-ts/msaai-540-mlops

The repository contains executed and refined notebooks under notebooks/, shared feature code under src/features.py with unit tests, requirements and CI workflow, exported figures under images/, and the ML system design document under docs/. housing.csv is intentionally not committed; data/README.md explains how to obtain it. This working file is the narrative project report that ties those artefacts together.

2. Dataset and Exploratory Analysis

2.1 Data Source and Dictionary

The project uses a single public source: California Housing (housing.csv, approximately 1.2 MB) derived from the 1990 U.S. census. It contains 20,640 rows (one per block group) and 10 columns. Data quirks that shaped the design include 207 rows (1%) with missing total_bedrooms, zero duplicate rows, 965 rows of median_house_value censored at $500,001, and 1,273 rows of housing_median_age capped at 52. Ocean proximity counts are <1H OCEAN (9,136), INLAND (6,551), NEAR OCEAN (2,658), NEAR BAY (2,290), and ISLAND (5). All rows are retained so evaluation reflects production conditions; censoring is reported as a limitation.

Table 2. Data dictionary (raw columns)

Column	Type	Description	Notes
longitude	float	Block group longitude	≈ −124.35 to −114; corr. lat −0.92
latitude	float	Block group latitude	≈ 32.54 to 42
housing_median_age	int	Median age of houses (years)	Capped at 52 (1,273 rows)
total_rooms	int	Total rooms in block group	Right-skewed; block size
total_bedrooms	float	Total bedrooms	207 missing (1%)
population	int	Residents	Right-skewed
households	int	Households	Corr. 0.98 with bedrooms
median_income	float	Median household income	Strongest predictor; corr. 0.69
ocean_proximity	category	Distance band to ocean/bay	Five classes; ISLAND n=5
median_house_value	int	Target (USD)	Censored at $500,001 (965 rows)

2.2 Data Quality and Ingestion Design

The notebook loads housing.csv, prints shape (20,640 × 10), and checks missing values, duplicates, category counts, and rows at each cap. In the production design the raw file is uploaded to a versioned, server-side-encrypted S3 bucket under a dated prefix (e.g., raw/housing/2026-10-01/housing.csv). Pipeline steps never overwrite raw bytes: each run reads a named version and writes outputs to a run-specific prefix so any registered model can be traced to exact training inputs.

Table 3. Data quality checks (pipeline gates)

Check	Rule	Action on failure
Schema	All 10 columns with expected types	Fail the pipeline
Row count	Within ~20% of previous version	Fail and alert

Missing values	Only total_bedrooms may be null; <5%	Fail and alert
Duplicates	Zero duplicate rows	Drop and log, or fail
Value ranges	Lat 32–42.5; lon −124.5 to −114; positives	Fail and alert
Categories	ocean_proximity in five known values	Warn new; fail malformed

2.3 Exploratory Findings

EDA drove preprocessing and feature choices. The target is right-skewed with a sharp peak at the $500,001 cap. Count features are heavily right-skewed and linearly correlated with each other (0.86 to 0.98), mostly reflecting block-group size. median_income is the strongest linear correlate of price (0.69). Location effects are strong and non-linear: coastal, Bay Area, and Los Angeles blocks are far more expensive than inland ones; INLAND has the lowest median price and ISLAND the highest.

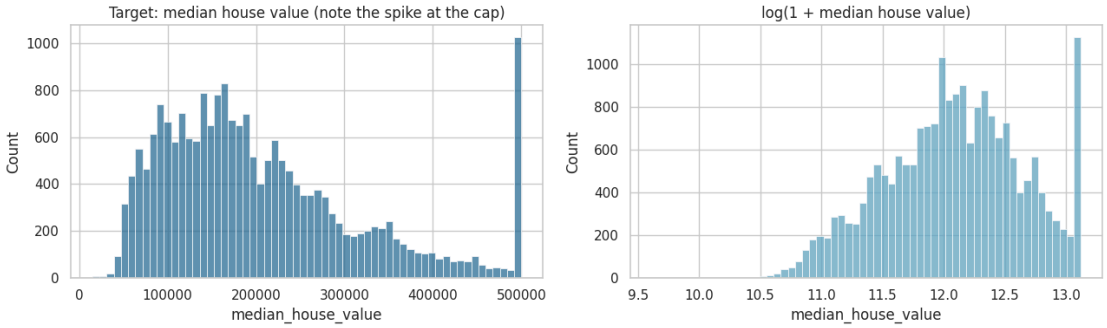


Figure 1. Target distribution with spike at the $500,001 cap (left) and log-transformed target (right).

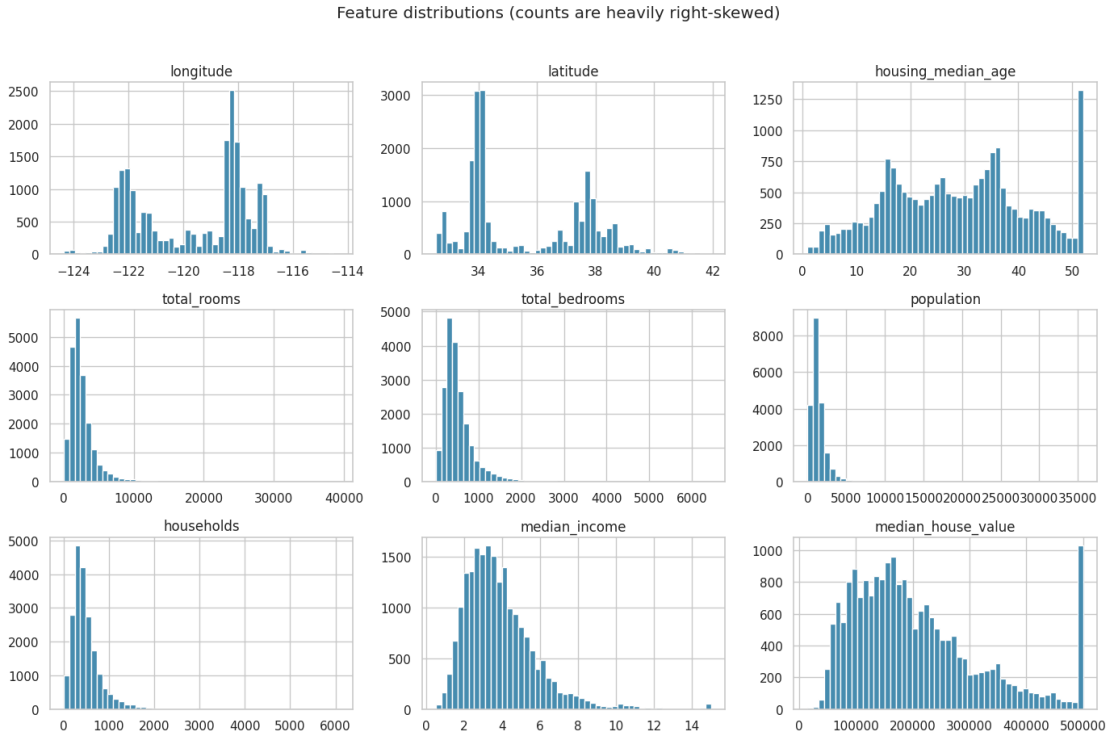


Figure 2. Feature distributions. Count features are heavily right-skewed.

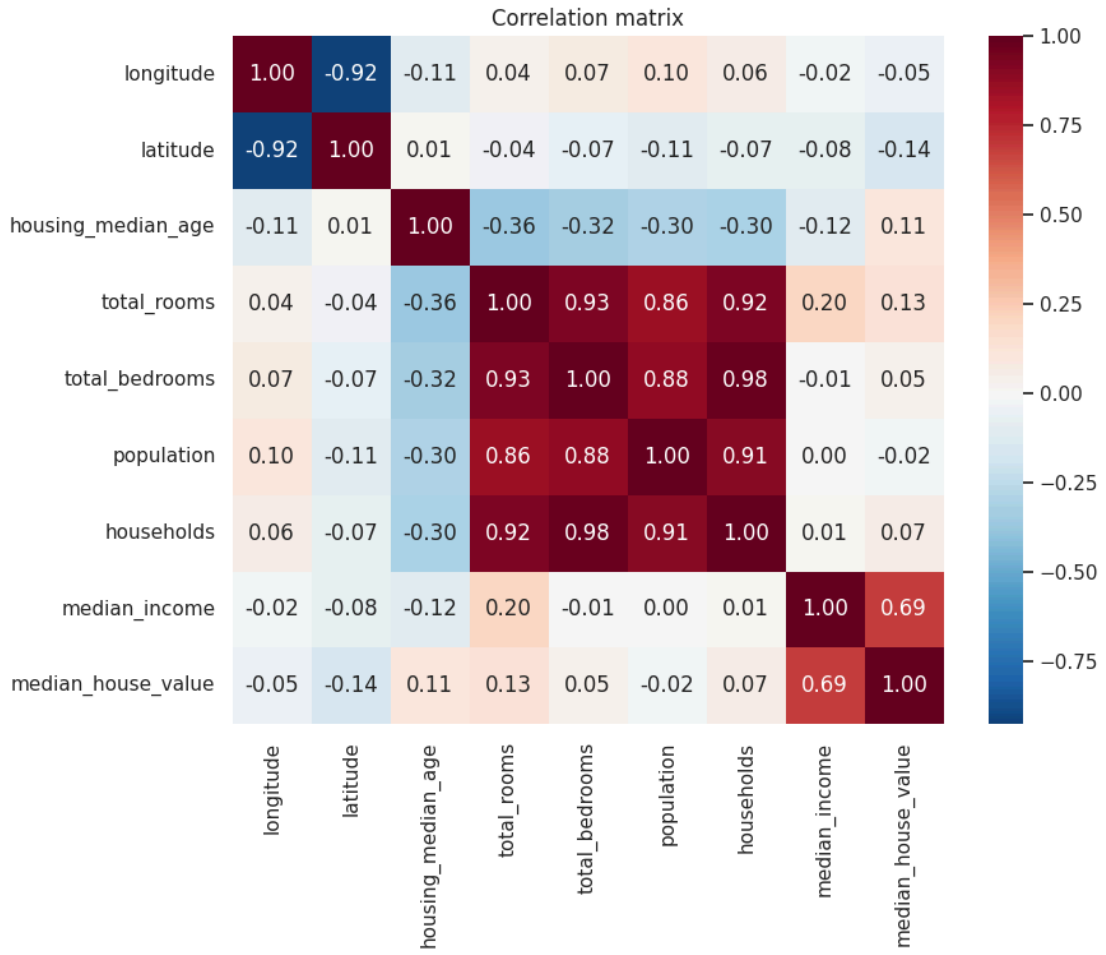


Figure 3. Correlation matrix. median_income correlates 0.69 with price; counts correlate with each other.

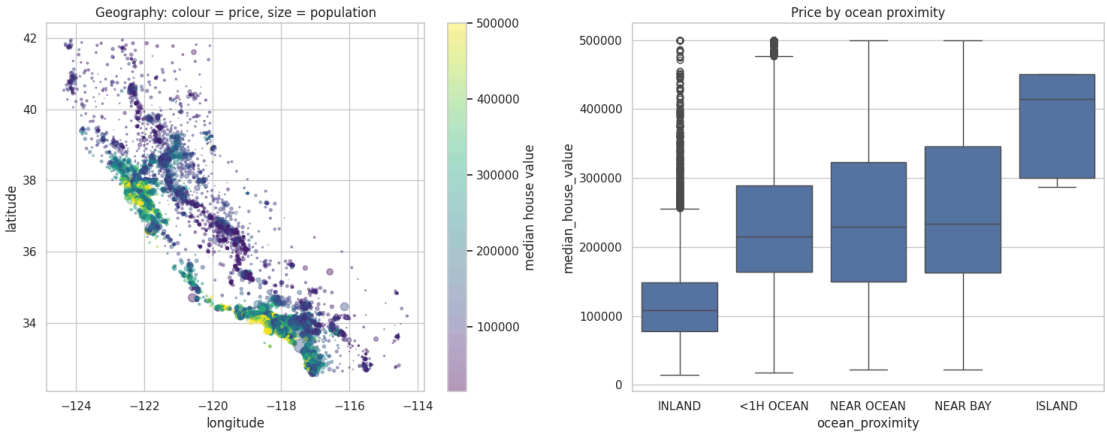


Figure 4. Geography (colour = price, size = population) and price by ocean proximity.

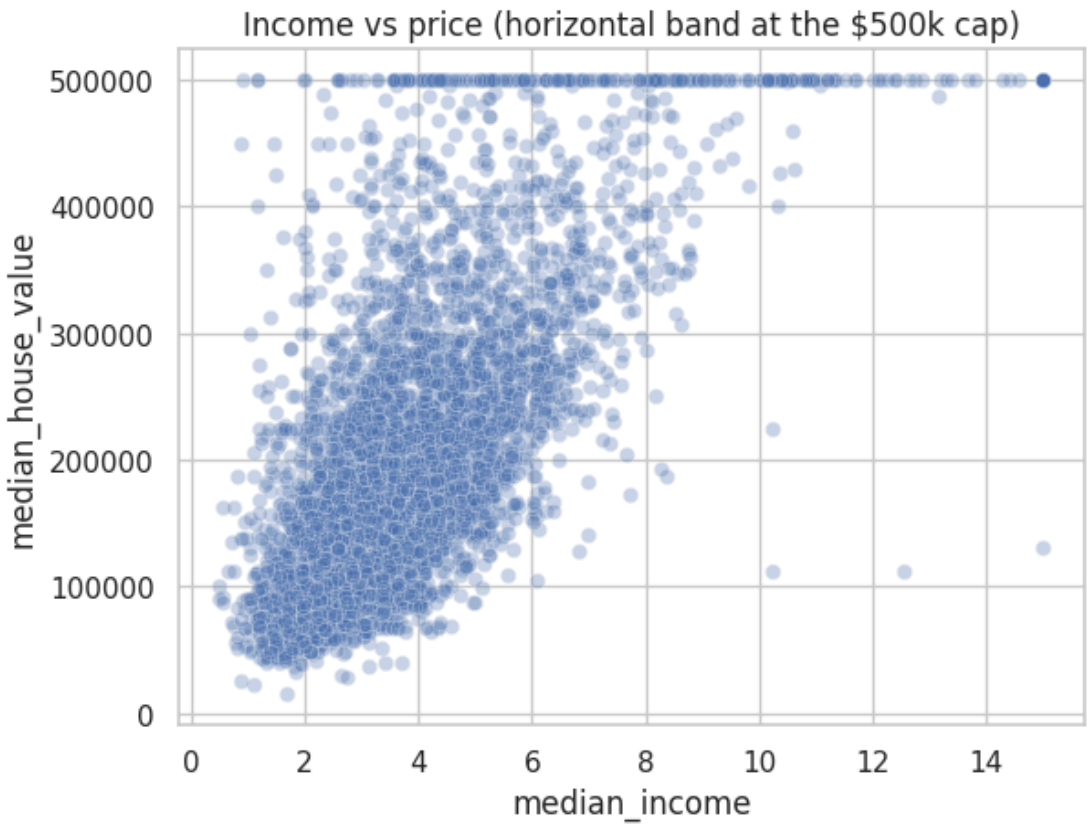


Figure 5. Income versus price, with a horizontal band at the $500k cap.

2.4 Preprocessing Design

Two preprocessing pipelines are used because linear and tree models have different needs. Keeping preprocessing inside a scikit-learn Pipeline means the exact same transformations apply during training, cross-validation, testing, and (in production) serving. An 80/20 train/test split is stratified by income bands (16,512 / 4,128) so both sets share the same income mix. The test set is carved off once and never used for tuning.

Table 4. Preprocessing by model family

Step	Linear models	Tree-based models	Reason
Missing values	Median imputation	Median imputation	Robust to skew; fit on train only
Log transform (log1p)	Yes, skewed counts/ratios	No	Linear models need nearer-symmetric inputs
Scaling	Standard scaler	No	Needed for linear; irrelevant to trees
Categorical encoding	One-hot (unknown ignored)	One-hot (unknown ignored)	Handles rare ISLAND / unseen

Decisions considered and rejected included dropping capped rows (would inflate scores), dropping rows with missing bedrooms (median imputation retains them without leakage), mean

imputation (skew-sensitive), and removing correlated raw counts (trees can ignore them; ratios already capture useful signal).

3. Feature Engineering

3.1 Engineered Feature Catalogue

Six engineered features are added to the nine raw predictors, yielding 15 model inputs. Three ratios—rooms_per_household, bedrooms_per_room, and people_per_household—remove block-group size effects. Three geographic features—haversine distances to Los Angeles, San Francisco, and the nearest major population centre—encode urban proximity more smoothly than raw coordinates. All are deterministic row-wise calculations using fixed city coordinates, so they introduce no cross-row leakage.

Table 5. Engineered features

Feature	Definition	Why added	Corr. with price
rooms_per_household	total_rooms / households	Removes block size	0.152
bedrooms_per_room	total_bedrooms / total_rooms	Dense vs larger homes	−0.256
people_per_household	population / households	Crowding indicator	−0.024
dist_la_km	Haversine to Los Angeles	LA market pull	−0.130
dist_sf_km	Haversine to San Francisco	Bay Area market pull	−0.031
dist_nearest_city_km	Min distance to major centres	Smooth urban proximity	−0.433

Engineered features proved useful in both correlation and permutation importance: dist_nearest_city_km is the strongest negative correlate of price ($r = -0.433$). Raw median_income remains the strongest single feature (≈0.688).

3.2 Rationale and Leakage Control

Three considerations drove the design. First, EDA showed count features mostly measure how large a block group is, not how valuable its homes are. Second, explicit city distances give trees a smooth urban-proximity signal they would otherwise approximate with many coordinate splits. Third, every engineered feature uses only that row (or fixed coordinates), so computing them before the split cannot leak information. Anything that learns from data—imputation values, scaling parameters, category lists—is fitted inside the pipeline on training data only, including within each cross-validation fold.

3.3 Shared Feature Code

To prevent training/serving skew, the same feature function is intended for notebooks, unit tests, and future endpoint inference code. In the repository (https://github.com/bharath-ts/msaai-540-mlops), src/features.py exposes add_features() and

haversine_km(), covered by tests under tests/. GitHub Actions runs those tests on push and pull request.

4. Model Development and Training

4.1 Candidate Models

Six models span increasing complexity so each step up can be justified by measured gain rather than assumed: Linear Regression (baseline), Linear Regression with a log-transformed target, Decision Tree, Random Forest, Gradient Boosting (scikit-learn), and XGBoost.

Table 6. Candidate models and rationale

Model	Role	Rationale
Linear Regression	Baseline	Fast, interpretable benchmark
Linear Regression (log target)	Variant	Tests whether log price fixes skew (it did not)
Decision Tree	Non-linear baseline	Non-linearity without ensembling
Random Forest	Bagging ensemble	Reduces tree variance
Gradient Boosting	Boosting ensemble	Sequential error correction
XGBoost	Optimised boosting	Regularised, fast; best overall

4.2 Train/Validation Protocol

Model selection and hyperparameter search use 5-fold cross-validation on the training set only, with a fixed random state. Preprocessing is fitted inside each CV fold. All models are ranked on the same metrics (MAE, RMSE, R^2), with RMSE as the primary selection metric. The held-out test set is reserved for Chapter 5.

4.3 Hyperparameter Search

XGBoost and Random Forest were tuned with RandomizedSearchCV. Randomised search was chosen over a full grid because the XGBoost space alone has 972 combinations; 15 were sampled for XGBoost and a smaller budget for Random Forest, with refit on the full training set.

Table 7. Hyperparameter search spaces

Model	Parameter	Values searched
XGBoost	n_estimators	300, 500, 800
XGBoost	learning_rate	0.03, 0.05, 0.08, 0.1
XGBoost	max_depth	4, 6, 8
XGBoost	min_child_weight	1, 5, 10
XGBoost	subsample	0.7, 0.85, 1.0
XGBoost	colsample_bytree	0.6, 0.8, 1.0
Random Forest	n_estimators	150, 250
Random Forest	max_depth	None, 25
Random Forest	min_samples_leaf	1, 2, 3
Random Forest	max_features	0.4, 0.6

Selected XGBoost: 800 estimators, subsample 0.85, min_child_weight 5 (CV RMSE 43,664 vs 44,359 untuned). Selected Random Forest: 150 estimators, min_samples_leaf 1, max_features 0.4 (CV RMSE 46,209 vs 46,452 untuned). The small gains indicate defaults were already near-optimal; further accuracy would come from better data or features rather than more tuning.

4.4 Cross-Validation Results

On the training set, untuned XGBoost led the six-model comparison. Tuning improved XGBoost CV RMSE from 44,359 to 43,664 and Random Forest from 46,452 to 46,209.

Table 8. Five-fold cross-validation (training set)

Model	CV MAE	CV RMSE	CV RMSE std	CV R^2
XGBoost	28,460	44,359	1,069	0.853
Random Forest	29,768	46,452	755	0.839
Gradient Boosting	30,580	46,652	990	0.837
Decision Tree	38,815	58,032	959	0.748
Linear Regression	45,693	63,551	890	0.698
Linear Regression (log target)	45,487	78,010	3,666	0.544

5. Evaluation and Results

5.1 Held-Out Test Performance

On the 4,128-row test set, tuned XGBoost was best: MAE $26,882, RMSE $41,441, R^2 0.868. It explains about 87% of variance in median house value and is off by about $27k on average (13.0% of mean price). Ensemble trees clearly beat the linear baseline (R^2 0.85–0.87 vs 0.71). Ranking matched between CV and test; test RMSE (41,441) is close to tuned CV RMSE (43,664), indicating no overfitting. Tuning lowered test RMSE by about 1.4% versus untuned XGBoost. The log-target linear model performed worst.

Table 9. Held-out test results (4,128 block groups)

Model	MAE ($)	RMSE ($)	R^2
XGBoost (tuned)	26,882	41,441	0.868
XGBoost	27,654	41,991	0.865
Random Forest (tuned)	28,335	43,565	0.854
Random Forest	28,772	44,340	0.849
Gradient Boosting	30,010	44,508	0.848
Decision Tree	38,240	55,881	0.760
Linear Regression	45,292	61,938	0.706
Linear Regression (log target)	44,723	83,531	0.465

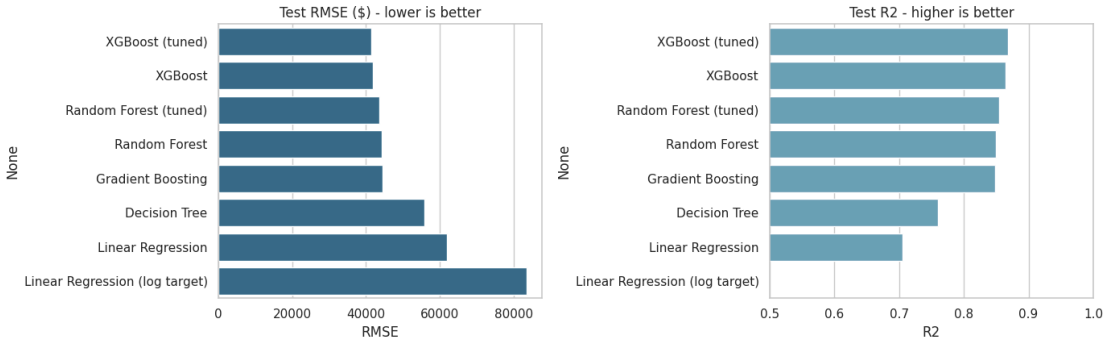


Figure 6. Test RMSE (lower better) and R² (higher better) by model. Tuned XGBoost is best.

5.2 Success Criteria

Table 10. Targets, baseline, and achieved results

Metric	Baseline (linear, test)	Target	Achieved (tuned XGBoost)	Met?
R^2	0.706	≥ 0.85	0.868	Yes
MAE	$45,292	≤ 15% of mean price	$26,882 (13.0%)	Yes
RMSE	$61,938	Lower than baseline	$41,441	Yes
CV vs test gap	n/a	Test within a few % of CV	CV 43,664 → test 41,441	Yes
RMSE on capped rows	n/a	Report separately	$80,827 (n=179) vs $38,717 below	Reported

Note. Tuned XGBoost improves MAE by 41% and RMSE by 33% versus the linear baseline.

5.3 Error Analysis and Feature Importance

Error concentrates on censored rows: RMSE is $38,717 below the price cap but $80,827 on the 179 capped test rows—expected, because the model cannot learn true values above $500,001. Residuals centre near zero and are roughly symmetric, with a visible diagonal streak of capped points. Predicted-versus-actual tracks the diagonal closely except for the vertical band at $500,001.

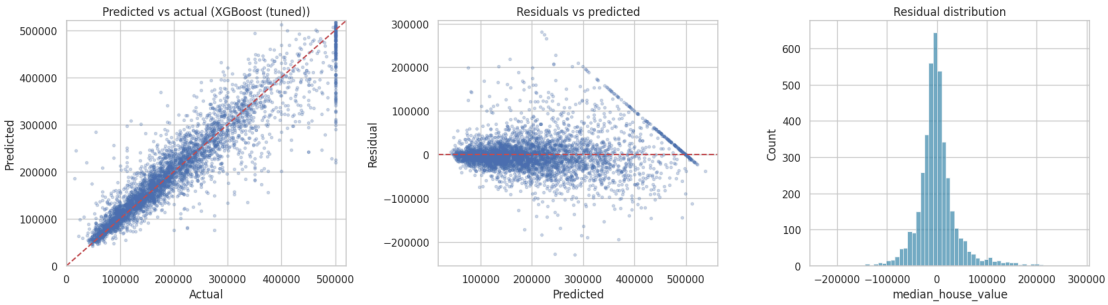


Figure 7. Predicted vs actual, residuals vs predicted, and residual distribution for tuned XGBoost.

Permutation importance (approximate values from the chart) ranks median_income first (~$30,500 RMSE increase), then longitude, latitude, ocean_proximity, dist_nearest_city_km, and

people_per_household. Four of six engineered features rank in the top nine. Raw count features contribute near zero once ratios exist.

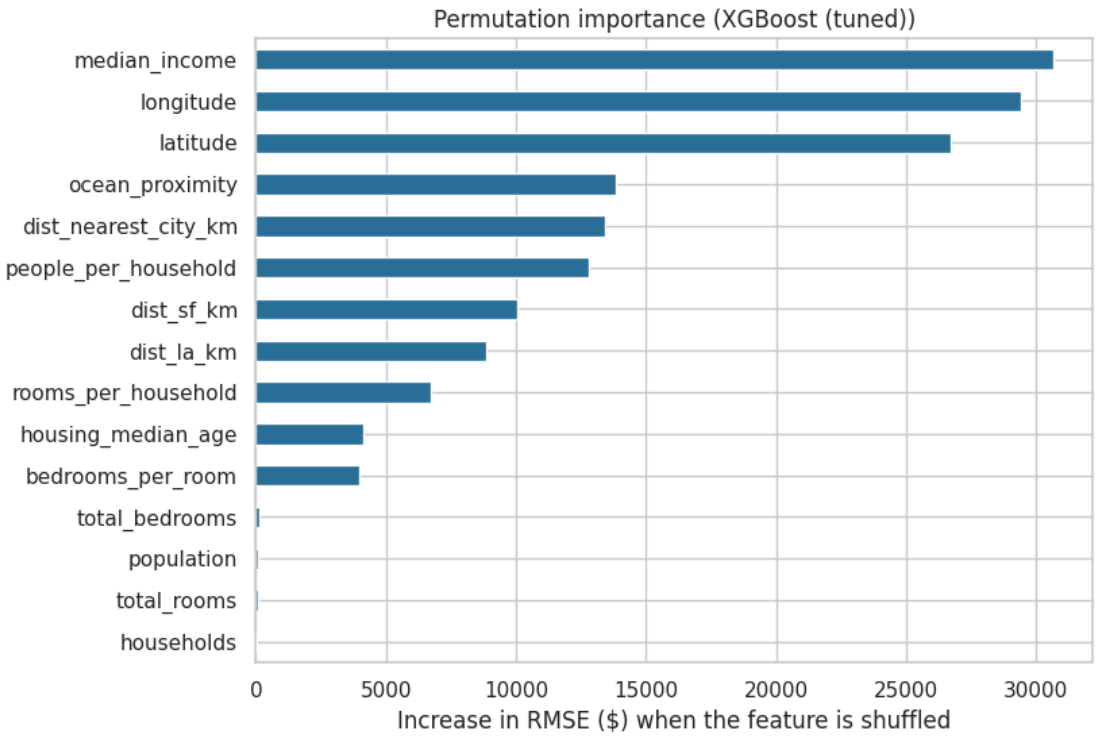


Figure 8. Permutation importance for tuned XGBoost (increase in RMSE when a feature is shuffled).

Table 11. Permutation importance (approximate)

Rank	Feature	Approx. RMSE increase ($)
1	median_income	30,500
2	longitude	29,500
3	latitude	26,500
4	ocean_proximity	13,800
5	dist_nearest_city_km	13,500
6	people_per_household	12,800
7–9	dist_sf_km, dist_la_km, rooms_per_household	10,000 / 8,800 / 6,700
10–11	housing_median_age, bedrooms_per_room	4,100 / 4,000
12–15	raw counts	Near zero

5.4 Sample Prediction

For a Bay Area neighbourhood example (longitude −122.25, latitude 37.85, housing_median_age 30, total_rooms 2500, total_bedrooms 500, population 1200, households 450, median_income 5.5, ocean_proximity NEAR BAY), the tuned model estimated median house value at $268,218. This example is the reference case for the inference contract in Chapter 9.

6. ML System Architecture

6.1 End-to-End Workflow

The project has nine notebook stages: data collection, data cleaning, exploratory data analysis, feature engineering, train/test split, model training, model comparison and tuning, evaluation, and price prediction. Stages 1–4 prepare data; stages 5–9 cover modelling and results. Each stage maps to SageMaker services in the production design.

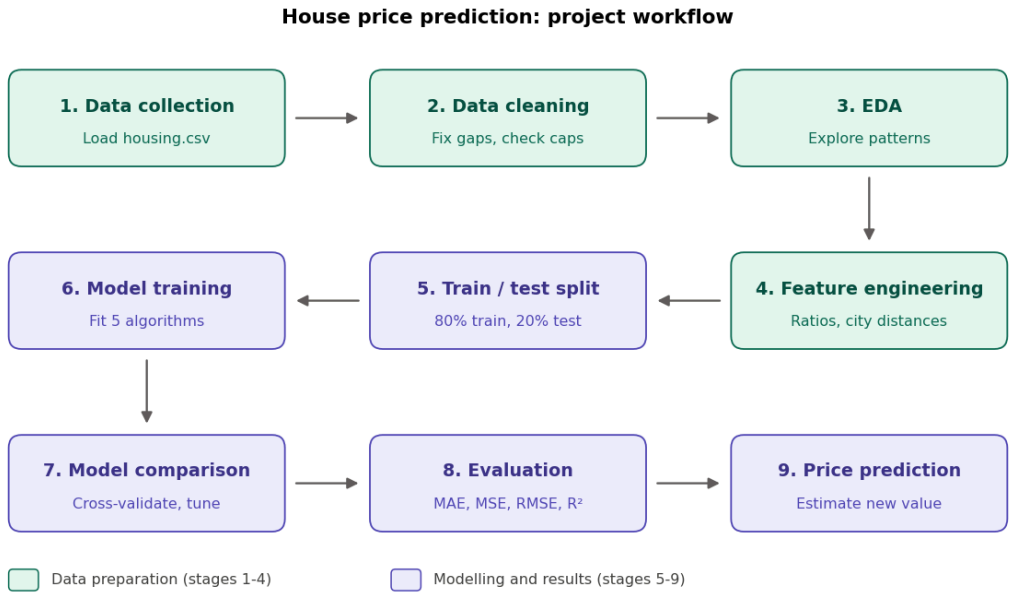


Figure 9. Notebook project workflow (stages 1 to 9).

6.2 Notebook versus SageMaker Mapping

Table 12. Workflow stage mapping

Workflow stage	Notebook implementation	SageMaker / AWS component
1. Data collection	Load housing.csv	S3 (versioned, encrypted)
2. Data cleaning	Null/duplicate/cap checks	Processing job + validation rules
3. EDA	Plots and summaries	Studio / Data Wrangler
4. Feature engineering	add_features()	Processing → Feature Store
5. Train/test split	Stratified 80/20	Processing channels
6. Model training	sklearn / XGBoost pipelines	Training jobs (script mode)
7. Comparison and tuning	5-fold CV; randomised search	Automatic Model Tuning
8. Evaluation	MAE/MSE/RMSE/R²	Eval report + quality gate
9. Prediction	predict_price()	Endpoint + Batch Transform
Versioning	Git commits	Model Registry
Monitoring	N/A in notebook	Model Monitor, Clarify, CloudWatch
Orchestration	Manual / local runs	Pipelines + CI/CD trigger

6.3 Implemented versus Design-Only Components

Clarity on status is essential for grading and for the demonstration video. The following summary is the authoritative status for this working file.

Table 13. Implementation status matrix

Component	Status	Evidence
Data load, EDA, features, models, metrics	Implemented	Notebooks 01/02; figures in images/
Shared feature module + unit tests	Implemented	src/features.py; tests/; GitHub Actions CI
predict_price() reference inference	Implemented	Notebook Stage 9
Design documentation	Implemented	docs/ml-system-design.md; this report
Versioned S3 raw store	Design-only	Described; not required for notebook results
SageMaker Feature Store groups	Design-only	Console evidence for demo (planned)
SageMaker Training / AMT	Design-only	Maps from local CV + randomised search
Model Registry + approval	Design-only	Quality gate $R^2 \geq 0.85$ specified
Real-time endpoint / Batch Transform	Design-only	Inference contract defined
Model Monitor / Clarify / CloudWatch	Design-only	Thresholds proposed
SageMaker Pipelines full DAG	Design-only	Nine-stage DAG specified
GitHub Actions (pytest)	Implemented	https://github.com/bharath-ts/msaai-540-mlops/actions

7. Data Platform and Feature Store

In production, raw housing.csv lands in a versioned, encrypted S3 prefix and is never mutated in place. Processing jobs validate schema and ranges, compute engineered features, and write to two Feature Store groups so training and inference share definitions. This chapter expands the data platform beyond the notebook's local file load.

Ingestion design. The raw file is uploaded under a dated prefix such as raw/housing/2026-10-01/housing.csv. Each pipeline run reads a named version and writes outputs to a run-specific prefix so any registered model can be traced to exact training bytes. There is no streaming ingestion in the current static public dataset; a future live feed would schedule new files into the same bucket structure and trigger the CI/CD DAG.

Table 14. Feature Store design

Feature group	Contents	Record ID	Event time	Stores
housing-raw-features	Nine raw predictors	block_group_id	Ingestion timestamp	Offline + online

housing-engineered-features	Six engineered features	block_group_id	Computation timestamp	Offline + online

The offline store supports point-in-time-correct training datasets and auditing. The online store lets a real-time endpoint look up features by identifier. Because median_house_value is a label, not a feature, it is kept out of the feature groups and joined only when a training set is built. This separation reduces accidental leakage into serving paths.

Feature Store also reinforces the shared-code discipline from Chapter 3: engineered ratios and distances are computed once in a Processing job using the same function as notebooks and unit tests, then served consistently to training channels and endpoints.

8. Training Pipelines and Model Registry

Implementation status. *Implemented in notebooks/repo: local training, CV, tuning, and metric reporting in notebooks | Design-only (SageMaker target): SageMaker Training, Automatic Model Tuning, evaluation step, Model Registry with PendingManualApproval*

The production training path mirrors the notebook protocol: validate data, engineer features, stratified split, train candidates, tune top ensembles, evaluate on the held-out test channel, then apply a quality gate. SageMaker Training jobs (script mode) replace local sklearn fits; Automatic Model Tuning replaces RandomizedSearchCV while preserving the same search spaces and selection metric (CV RMSE).

Model Registry. Models that pass are registered as new versions in a package group with MAE, RMSE, R^2, training data version, and code commit attached. New versions start as PendingManualApproval and become Approved only after the quality gate and human approval. Only Approved versions may be deployed; the previous Approved version remains available for rollback.

Quality gate (must pass before registration/promotion): R^2 at least 0.85, and RMSE not materially worse than the current production model. Failure stops the DAG before registration, notifies the team, and leaves production unchanged.

Table 15. Proposed operational KPIs for the training/serving system

KPI	Proposed target	Source
Real-time endpoint p99 latency	Under 200 ms	CloudWatch
Endpoint error rate (5xx)	Under 0.1% of invocations	CloudWatch
Batch job completion	100% of scheduled runs succeed	Pipeline history
End-to-end retraining time	Under 30 minutes	SageMaker Pipelines
Drift alerts triaged	Within 1 business day	Model Monitor, SNS
Pipeline runs passing quality gate	Failing runs never reach production	Model Registry

Note. KPI values are design targets, not measured production telemetry in the current notebook phase.

9. Deployment (Real-Time and Batch)

Two serving patterns are specified. For interactive use, nine raw fields are posted to a real-time endpoint that applies the same feature function and preprocessing pipeline used in training, then returns predicted median house value and model version. For portfolio screening, Batch Transform reads a CSV of block groups from S3 and writes predictions back to S3. Sharing the pipeline object and feature function eliminates the most common cause of training/serving skew.

Inference contract (request body). Callers send raw fields only; engineered features are computed server-side:

{"longitude": -122.25, "latitude": 37.85, "housing_median_age": 30, "total_rooms": 2500, "total_bedrooms": 500, "population": 1200, "households": 450, "median_income": 5.5, "ocean_proximity": "NEAR BAY"}

The notebook returns $268,218 for this input. Malformed, out-of-range, or unknown-category requests are rejected with a clear error and are not scored.

Table 16. Serving options considered

Option	Best for	Trade-off	Decision
Real-time endpoint	Interactive low latency	Always-on cost	Interactive use + auto-scaling
Serverless inference	Spiky / infrequent traffic	Cold starts	Consider for low volume
Batch Transform	Bulk scoring	Not interactive	Screening and reports

Release strategy: deploy to staging; run smoke tests (Stage 9 example plus edge cases such as ISLAND); shift production traffic gradually with SageMaker deployment guardrails (canary or linear) and automatic rollback on alarms. The previous Approved configuration remains deployable. Cost controls include small instance types, auto-scaling, off-hours scale-down, and Batch Transform for large offline jobs. Security for the endpoint includes IAM authentication, VPC placement, CloudWatch logging, and encrypted capture of requests and responses for monitoring.

10. Monitoring, Drift, and Alerts

Three monitoring layers are specified. Infrastructure monitoring uses CloudWatch dashboards for invocations, latency (p50/p99), 4xx/5xx errors, and resource utilisation. Data quality monitoring compares live traffic to training baselines for schema violations, missing values, and distribution drift (especially median_income and location). When ground truth becomes available, model-quality jobs track MAE, RMSE, and R^2 against the test baseline and alert if error exceeds an agreed tolerance (for example, RMSE more than 15% above ≈ $41k). Capped-area error is

reported separately. SageMaker Clarify supports bias and attribution views by region and income band.

Table 17. Monitoring plan

Layer	Measured	Tool	Example threshold	Action
Infrastructure	Latency, errors, CPU/mem	CloudWatch	p99 >200 ms or 5xx >0.1%	Page; scale; roll back
Data quality	Schema, nulls, ranges	Model Monitor	Any schema violation	Alert; block; investigate
Data drift	Income/location shift	Model Monitor	Distance above baseline	Alert; consider retrain
Model quality	MAE/RMSE/R^2 vs truth	Model Monitor	RMSE >15% above ~$41k	Trigger retrain
Capped-area error	High-value band	Custom metric	Rising error >$400k preds	Flag; review
Bias / explainability	Error by region/income	Clarify	Material regional gap	Compliance review

Drift is the primary early warning because true median house values for new periods arrive slowly. The model is highly sensitive to income and location, so distribution shifts in those features are likely to harm accuracy before labels confirm it. Alerting publishes to SNS; on-call first checks infrastructure, then data quality, then triggers retraining if confirmed. If the new model fails the quality gate, production is left unchanged and the issue is escalated. Every incident and version change is retained for audit.

11. Security and Governance

Direct privacy risk is low because the dataset is public aggregated census statistics without personal identifiers. Residual risk rises if property-level data (addresses, ownership) are later joined; that extension would require a new privacy review. Historical prices may embed discriminatory patterns (including redlining). Latitude, longitude, and ocean proximity can act as proxies for protected attributes if used for credit or insurance. The model predicts block-group medians, not individual homes. Mitigations: screening-only use with human review for consequential decisions; document limits in a model card; retrain on current data before production; evaluate error by region and income band.

Table 18. Security checklist

Item	Status	Notes
Data contains no PII	Complete	Aggregated census statistics
Encryption at rest	Planned (SageMaker)	S3 / Feature Store SSE-KMS
Encryption in transit	Planned	HTTPS/TLS
Least-privilege IAM	Planned	Separate roles; no wildcards
Network isolation	Planned	VPC private subnets
Secrets management	Planned	No credentials in code

Audit logging	Planned	CloudTrail / CloudWatch
Model and data versioning	Complete in design	Registry + dated datasets
Endpoint input validation	Planned	Reject out-of-range requests
Reproducibility	Complete (notebook)	Fixed seed; train-only preprocessing

Table 19. Risk register (summary)

Risk	Category	L / I	Mitigation
Used to appraise homes or lend	Ethical	M / H	Non-goals; human review; restrict access
Location as proxy for protected groups	Bias	M / H	Clarify; review before consequential use
Unreliable predictions near price cap	Technical	H / M	Report capped-row error; flag
Data drift over time	Operational	H / H	Monitor baselines; scheduled retrain
Training/serving skew	Technical	M / H	Shared feature code + Feature Store
Stale 1990 training data	Data	H / H	Refresh before production

Table 20. Data classification

Data asset	Classification	Handling
Raw housing.csv	Public, aggregated	Versioned encrypted S3; read-only pipeline roles
Engineered features	Derived from public data	Feature Store; ML-role access
Model artifacts	Internal	Model Registry; IAM only
Prediction traffic	Internal	Monitoring capture; limited retention
Logs and metrics	Internal	CloudWatch; restricted access

12. Continuous Integration and Delivery (CI/CD)

Repository CI at https://github.com/bharath-ts/msaai-540-mlops already runs unit tests for feature engineering. The production CD path is designed as a SageMaker Pipelines DAG that can also be triggered by CodePipeline or by monitoring alerts. Unit tests and a light pipeline pass on a tiny sample catch errors early and cheaply before a full expensive run. Human approval remains on the registry release path.

Table 21. Pipeline stages

#	Stage	Gate or check
1	Data validation	Schema, ranges, nulls, categories
2	Processing / features	Feature unit tests; no null/inf ratios
3	Train/test split	Sizes and income mix

4	Training and tuning	CV completes
5	Evaluation	Metrics JSON written
6	Quality gate	$R^2 \geq 0.85$; RMSE not materially worse than production
7	Register model	Pending approval
8	Deploy staging	Smoke tests
9	Promote production	Alarms green during traffic shift

Table 22. Testing strategy

Test type	What it covers	When
Unit	Feature functions, zero-division, ranges	Every commit
Data	Schema, ranges, categories	Start of every pipeline run
Model	Metric thresholds; fixed example tolerance	After training
Integration	Endpoint accepts inference contract	After staging deploy
Regression	Not materially worse than production	Quality gate

Successful runs register lineage and promote via staging. Failed runs stop at the first gate: validation failures avoid training spend; quality-gate failures never register or deploy. The demonstration plan includes one successful DAG and one forced failure (corrupted data or a stricter threshold).

13. Discussion, Limitations, and Future Work

13.1 Interpretation of Model Behavior

Price depends non-linearly on income and geography. Tree ensembles capture interactions (for example high income near the coast) that a linear model cannot. Explicit city distances give trees a smooth urban-proximity signal. The small tuning gain suggests inductive bias and features matter more than exhaustive search for this dataset size. Identical CV and test rankings, with test RMSE slightly better than CV RMSE, support generalisation under this split protocol.

13.2 Effect of Target Censoring

Analogous to class imbalance in classification, censoring at $500,001 systematically distorts evaluation if ignored. Aggregate RMSE mixes well-behaved uncapped rows with structurally unlearnable capped rows. Reporting both bands makes the limitation visible and prevents overstating readiness for high-value areas.

13.3 Reproducibility

A fixed random state governs the train/test split, CV folds, and randomised search. Pipelines bundle preprocessing with the estimator. Production design adds data version IDs, code commits,

pinned library versions, and Model Registry lineage. Code lives at https://github.com/bharath-ts/msaai-540-mlops.

13.4 Methodological Consideration: Random Split Versus Spatial Generalisation

A random split can place adjacent block groups in both train and test sets, so reported accuracy may be slightly optimistic for predicting an entirely new region. Spatial cross-validation is recommended before claiming geographic transfer. Absolute dollar errors are also historical (1990 dollars); production use requires retraining on current data.

13.5 Limitations

- Target censored above $500,001; model cannot learn the upper tail.
- Data from 1990 census; markets and demographics have changed.
- Small dataset (~1.2 MB); larger geographies need distributed processing and online Feature Store.
- ISLAND has only five rows.
- Metrics are block-group level, not property level.
- Operational KPIs are design targets, not yet measured from a live endpoint.
- Most SageMaker components remain design-only pending the AWS demonstration.

13.6 Future Work

- Spatial cross-validation and error maps by region.
- Expanded hyperparameter search or alternative tabular learners.
- Additional features (schools, transit, recent sales) with privacy review if property-level.
- Prediction intervals / quantile regression for uncertainty.
- Censored-regression treatment of the $500,001 cap.
- Implement and demonstrate the full SageMaker CI/CD path (success and forced-failure runs).

14. Conclusion

Tuned XGBoost is statistically and practically stronger than linear and single-tree baselines on this task, with no sign of overfitting under the stated protocol: R^2 0.868 and mean absolute error about $27k (13% of mean price). The largest modelling weakness is target censoring, where error nearly doubles. Wrapped in a SageMaker-oriented design—versioned data, Feature Store, registry quality gates, monitoring, and gated CI/CD—the solution is positioned as a repeatable first-pass valuation system, not a stand-alone appraisal tool. Remaining work is to refresh the data, validate spatial generalisation, and complete the production demonstration evidence. Project artefacts: https://github.com/bharath-ts/msaai-540-mlops.

15. References

Amazon Web Services. (n.d.). Amazon SageMaker Developer Guide. https://docs.aws.amazon.com/sagemaker/

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 785–794). https://doi.org/10.1145/2939672.2939785

Google Cloud. (2024). MLOps: Continuous delivery and automation pipelines in machine learning. Cloud Architecture Center.

Huyen, C. (2022). Designing machine learning systems: An iterative process for production-ready applications. O'Reilly Media.

Kreuzberger, D., Kühl, N., & Hirschl, S. (2023). Machine learning operations (MLOps): Overview, definition, and architecture. IEEE Access.

Nugent, C. (n.d.). California housing prices [Data set]. Kaggle. https://www.kaggle.com/datasets/camnugent/california-housing-prices

Pace, R. K., & Barry, R. (1997). Sparse spatial autoregressions. Statistics & Probability Letters, 33(3), 291–297.

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Sculley, D., et al. (2015). Hidden technical debt in machine learning systems. In Advances in Neural Information Processing Systems, 28.

Team repository. (2026). msaai-540-mlops. GitHub. https://github.com/bharath-ts/msaai-540-mlops

Appendix A. Demonstration Evidence Checklist

Table 23. Evidence to capture for the demonstration

Component	Evidence	Source	Status
Feature groups	Feature group list / sample record	AWS console	Design-only / Planned
Infra monitoring	CloudWatch endpoint dashboard	AWS console	Design-only / Planned
Data/model monitoring	Model Monitor reports	AWS console	Design-only / Planned
CI/CD success run	Pipeline graph all green	AWS console	Design-only / Planned
CI/CD failed run	Stopped at validation or quality gate	AWS console	Design-only / Planned
Model registry	Versions with metrics	AWS console	Design-only / Planned

Endpoint / batch output	Prediction response or S3 output	Notebook + console	Notebook done; AWS planned
Evaluation findings	Metrics, plots, importance	Notebook Figures	Implemented
GitHub CI	pytest green on main/PR	GitHub Actions	Implemented

Appendix B. Glossary

Term	Meaning
Block group	Smallest geography for published census statistics; one dataset row
Censoring	Values above a limit recorded as the limit ($500,001 here)
Cross-validation	Repeated train/validate splits to estimate generalisation
Data drift	Input distribution shift vs training data
Data leakage	Test or future info influencing training
Feature Store	Managed store for versioned training/serving features
MAE / RMSE / R^2	Mean absolute error / root mean squared error / variance explained
Model Registry	Catalogue of model versions, metrics, lineage, approval
Permutation importance	Performance drop when one feature is shuffled
Training/serving skew	Mismatch between train-time and serve-time features

Appendix C. Key Repository Paths

Root: https://github.com/bharath-ts/msaai-540-mlops

- notebooks/01_house_price_regression.ipynb — executed run; source of reported metrics
- docs/ml-system-design.md — detailed system design
- images/ — exported EDA and evaluation figures
- data/README.md — how to obtain housing.csv (not committed)

Appendix D. Summary of Key Implementation Artefacts

- Notebook pipeline stages 1–9 (load → clean → EDA → features → split → train → tune → evaluate → predict).
- Shared feature function for ratios and haversine distances (training and serving).
- scikit-learn Pipelines binding preprocessing to each estimator.
- RandomizedSearchCV for XGBoost and Random Forest.
- predict_price() reference inference function.
- Design artefacts: Feature Store groups, Model Registry gates, Monitor/Clarify plan, Pipelines DAG.